Reviewer Pack — Minimal Order of Sheaves over Affine Schemes and Resolution …

Entry 4b2c8aa24f21 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:20:04 UTC

Minimal Order of Sheaves over Affine Schemes and Resolution Proof Width Correlation

Entry ID: `4b2c8aa24f21` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 02:20:04 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Sheaf Theory) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

> For every Tseitin formula φ_G with n variables associated with a d-regular graph G, the minimal order of sheaves over its affine scheme, denoted by ord(min_sheaf(G)), is linearly correlated with its resolution proof width w(φ_G), such that ord(min_sheaf(G)) = Θ(w(φ_G)).

Rationale (proposer's reasoning):

> Sheaf theory provides a natural framework for studying algebraic structures in geometry, which may reveal hidden complexity-theoretic properties. The minimal order of sheaves over an affine scheme corresponds to the simplest non-trivial structure that can be defined on G. If this invariant is correlated with resolution proof width, it could expose new insights into the hardness of satisfiability problems.

Taxonomy category: `Sheaf_theory_resolution_proof_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f9888f42bf83fd79`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all Tseitin formulas φ_G with n variables (n ≤ 40), the ratio of ord(min_sheaf(G)) to w(φ_G) falls within a constant factor of Θ(n). This criterion is met if the mean ratio across 30 random seeds falls within ±10% of the expected value.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order of sheaves" AND "resolution proof width" IN title - "affine scheme" AND Tseitin formula IN abstract - "Sheaf Theory" AND "Complexity Theory" AND resolution proof complexity IN keywords

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
from itertools import combinations

def generate_random_graph(n, d=3):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")
    edges = set()
    for i in range(d):
        nodes = list(range(n))
        random.shuffle(nodes)
        for j in range(1, d):
            edge = (nodes[0], nodes[j])
            if edge not in edges and edge[::-1] not in edges:
                edges.add(edge)
    return {i: [] for i in range(n)}, edges

def tseitin_formula(graph, n):
    clauses = []
    for node in range(n):
        literals = [f'x{i}' for i in range(node * d + 1, (node + 1) * d)]
        clause = ['~'] + literals
        clauses.append(clause)
        for literal in literals:
            clauses.append([literal])
        for i in range(len(literals)):
            for j in range(i + 1, len(literals)):
                clauses.append(['~', literals[i], '~', literals[j]])
    return clauses

def resolution_width(clauses):
    queue = set()
    for clause in clauses:
```

```
            if len(clause) == 1:
                queue.add(clause[0])
            else:
                queue.add(tuple(sorted(clause)))
        while True:
            new_clauses = []
            found_resolvent = False
            for c1, c2 in combinations(queue, 2):
                resolvents = set()
                for l1 in c1:
                    if '~' + l1 in c2:
                        resolvents.add(tuple(sorted([x for x in c1 if x != l1] + [x for x in c2 if x !
                if not resolvents:
                    continue
                found_resolvent = True
                new_clauses.extend(resolvents)
            if not found_resolvent:
                break
            queue.update(new_clauses)
        return max(len(c) for c in queue)

    def min_sheaf_order(graph):
        n, _ = graph
        order = 0
        for node in range(n):
            neighbors = graph[node]
            if len(neighbors) > order:
                order = len(neighbors)
        return order

    def run_trial(seed: int) -> dict:
        random.seed(seed)
        results = []
        for n in [5, 10, 15, 20, 30, 40]:
            graph = generate_random_graph(n)
            clauses = tseitin_formula(graph, n)
            sheaf_order = min_sheaf_order(graph)
            width = resolution_width(clauses)
            results.append((sheaf_order, width))
        if not results:
            return {
                "metric_name": "Ratio of Sheaf Order to Resolution Width",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": 0,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }
        ratio = sum(s / w for s, w in results) / len(results)
        return {
            "metric_name": "Ratio of Sheaf Order to Resolution Width",
            "metric_value": ratio,
            "instances_tested": len(results),
            "n_max": max(n for _, n in results),
            "conjecture_holds": abs(ratio - n) / n < 0.1,
            "counterexample": ""
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(trial_result)

    if not all(r["instances_tested"] > 0 for r in results):
        print("RESULT: INCONCLUSIVE reason=insufficient_data n_tested=<k>")
    else:
        mean_ratio = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_ratio} std=<y> support_fraction={support_fractior
        else:
            first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), N
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}"
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_013bd294.py", line 109, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_013bd294.py", line 80, in run_trial
    graph = generate_random_graph(n)
            ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_013bd294.py", line 19, in generate_rand
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that the test can complete successfully.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15936
2	propose	ollama_remote	glm4:latest	0	0	12121
3	propose	ollama_remote	glm4:latest	0	0	20706
4	preregistration	ollama_remote	glm4:latest	0	0	9703
5	novelty	ollama_remote	glm4:latest	0	0	9214
6	novelty	ollama_remote	glm4:latest	0	0	9052
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16845
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10639
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16085
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12524
11	judge	ollama_remote	glm4:latest	0	0	11533

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 144358 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/4b2c8aa24f21.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b2c8aa24f21.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4b2c8aa24f21.tar.gz` (if generated)